



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto AnzenCore

Curso: Calidad y Pruebas de Software

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

Arocutipa Arocutipa, Gian Franco (2023076790)

Perez Peralta, Fabrizio Salvador Elias (2023077476)

**Tacna – Perú
2026**

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

Documento de Estándares de Programación

Versión 1.0

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

Historia de Revisión

Historial de revisiones				
Item	Fecha	Versión	Descripción	Equipo
1	04/07/206	1.0		FSEPP - GFAA

Tabla de Contenidos

Estándares de Programación	4
1. OBJETIVO	4
2. DECLARACIÓN DE VARIABLES	5
2.1 Descripción de la Variable.	5
2.2 Variables de Tipo Arreglo	5
3. Definición de datos y funciones	5
3.1 Tipo de datos	5
3.2 Declaración de variables	5
3.3 Declaración de funciones	5
3.4 Control de versiones de código fuente	5
4. Procedimientos y Funciones definidos por el Usuario.	5
5. Beneficios	5
6. Conclusiones	5

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

Estándares de Programación

1. OBJETIVO

- Asegurar la validez de los datos de entrada mediante esquemas Pydantic en la capa de servicio, reforzados con restricciones CHECK directamente en la base de datos PostgreSQL/Supabase para los campos enumerados (status, severity, severity_max, export_format, component_type). AnzenCore no usa Triggers.
- Centralizar los procesos de análisis en clases de servicio Python (AnalysisService, ApkAnalyzer, ReportService) para evitar la duplicidad de código y asegurar que la lógica se ejecute siempre de manera coherente.
- Registrar timestamps de auditoría (created_at, started_at, finished_at, fecha) en cada análisis y reporte. Esto es vital para trazabilidad, depuración y para mantener un historial completo de los escaneos.
- Optimizar el código (límites de tamaño de archivo, exclusión de carpetas irrelevantes, ejecución en hilos con asyncio.to_thread) para que la API mantenga buena velocidad de respuesta bajo carga.
- Estructurar el sistema por capas (routes, services, models, schemas) con separación de responsabilidades. Esto permite corregir o añadir funcionalidades sin afectar al resto del sistema.
- Crear consultas Supabase dedicadas (get_all_apk_scans, get_apk_findings, get_apk_artifacts) para consolidar datos clave y facilitar el dashboard de métricas.

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

2. DECLARACIÓN DE VARIABLES

Se propone que la declaración de variables siga el estándar PEP 8 de Python, ajustándose al motivo para el que se requieran. La convención definida se establece tomando en consideración principalmente lo siguiente:

- El nombre debe ser descriptivo y en snake_case (minúsculas separadas por guion bajo), evitando abreviaturas crípticas. Ejemplos reales del proyecto: file_hash_sha256, severity_max, findings_count.

- Alcance de la variable

A medida que aumenta el tamaño del proyecto, también aumenta la utilidad de reconocer rápidamente el alcance de las variables. En Python esto se logra mediante convenciones de nombre en vez de prefijos: MAYUSCULAS para constantes de módulo, snake_case normal para variables locales y un guion bajo inicial para miembros/métodos privados de una clase.

Alcance	Prefijo	Ejemplo
Global (constante de módulo)	MAYÚSCULAS	SEVERITY_ORDER, TIPOS_VALIDOS
Local	snake_case	file_bytes, user_id
Privado (clase/módulo)	_snake_case	_hash_password, _build_response

Estructura	Descripción de la Variable
LONGITUD MÁX.	Sin límite estricto; se prioriza claridad sobre brevedad (PEP 8).
FORMATO	snake_case para variables/funciones, PascalCase para clases.

Proyecto Sistema de gestión de eventos		Versión: 1.0	
Documento de Estándares de Programación			
	EJEMPLO	analizar_apk (función), ApkAnalyzer (clase)	

2.1 Declaración de Variables Locales (SQL)

A. Variables de Identificación (Claves Primarias y Foráneas)

Variable	Tipo de Dato (Ejemplo)	Descripción	Módulo
id	uuid	Clave primaria (default gen_random_uuid()); en vulnerabilidades es integer (serial).	Todos
user_id	uuid (FK)	Clave foránea que referencia a usuarios.id.	Usuarios / Autenticación
scan_id	uuid (FK)	Clave foránea que referencia a apk_scans.id.	Análisis de APK
token_hash	text	Hash del token de un cliente de integración externa.	integration_clients

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

B. Variables de Datos Principales (Módulos Evento e Inscripción)

Utilizadas para registrar y actualizar la información específica de cada análisis (APK, código fuente, URL o repositorio).

Variable	Tipo de Dato (Ejemplo)	Descripción	Módulo
file_bytes	bytes	Contenido binario del archivo subido (APK o código fuente).	Análisis
file_name	str	Nombre del archivo analizado.	Análisis
tipo_analisis	str	Tipo de análisis solicitado: apk / codigo_fuente / url / repo_github.	Análisis
api_base_url	str	URL base de la API configurada en la extensión de VS Code.	Extensión VS Code
severity	str	Severidad del hallazgo: Critico/Alto/Medio/Bajo/Info.	Hallazgos

C. Variables de Control de Flujo, Lógica y Trazabilidad

Utilizadas para registrar y actualizar la información específica del negocio.

Variable	Tipo de Dato (Ejemplo)	Descripción	Módulo
status	str	Estado del análisis: pending / completed / failed.	apk_scans
error_message	str	Mensaje de error si el análisis falla.	Manejo de errores
findings_count	int	Cantidad de hallazgos detectados, usado para trazabilidad.	apk_scans
created_at / started_at	/ datetime	Marcas de tiempo de auditoría del	Auditoría

Proyecto Sistema de gestión de eventos		Versión: 1.0	
Documento de Estándares de Programación			
finished_at		análisis.	

D. Variables de Paginación y Filtrado (Para Reportes)

Utilizadas en las consultas del dashboard para filtrar el historial de escaneos por usuario y ordenar los resultados.

Variable	Tipo de Dato (Ejemplo)	Descripción	Módulo
user_id	UUID (str)	Filtra los escaneos/reportes por usuario propietario.	Historial
time_limit	datetime	Filtra usuarios con last_ping mayor a este valor (usuarios en línea).	Métricas
order_by=created_at desc	str	Orden de los resultados en el historial de escaneos.	Historial / Reportes

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

3. Definición de datos y funciones

Se deben utilizar los siguientes tipos de datos (Python / Supabase-Postgres) para asegurar la consistencia entre la API y la base de datos.

3.1 Tipo de datos

Uso	Tipo de Dato Estándar (Postgres/Supabase)	Razón de la Elección	Ejemplos de Campos
Identificadores	uuid (default gen_random_uuid())	Evita colisiones sin depender de un contador centralizado; excepción: vulnerabilidades.id es integer (serial).	id, user_id, scan_id
Valores booleanos	boolean	Tipo nativo de Postgres para estados binarios.	is_dangerous, exported, is_active
Fechas y tiempos	timestamp with time zone	Conserva la zona horaria; vital para la trazabilidad de escaneos.	created_at, started_at, finished_at, fecha, last_ping
Texto (sin límite fijo)	text	Postgres no penaliza 'text' frente a varchar(N); se evita fijar longitudes artificiales.	username, file_hash_sha256, summary, description
Enumerados	text + CHECK constraint	Restringe los valores permitidos directamente en la base de datos (equivalente a un ENUM).	status, severity, severity_max, export_format, component_type
Numéricos	bigint / integer	bigint para tamaños de archivo grandes; integer para contadores.	file_size_bytes, findings_count, source_line
Binario (en)	bytes (Python)	Contenido de archivos subidos	file_bytes

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

memoria, no persistido)	(APK/código fuente) mientras se procesa; no se guarda en la base de datos.
-------------------------	--

3.2 Declaración de Procedimientos Almacenados

Las clases de servicio Python (*Service) son la principal herramienta para encapsular el flujo de trabajo de análisis y reporte de AnzenCore, en ausencia de procedimientos almacenados en base de datos.

Regla	Descripción	Ejemplo (AnzenCore)
Nomenclatura	Los métodos de servicio usan snake_case y la convención verbo_sustantivo.	analizar_apk, analizar_codigo_fuente, create_report
Encapsulación	Cada acción se encapsula en una clase *Service, análoga a un procedimiento almacenado.	AnalysisService, ReportService, ApkScanService
Propósito	Cada método debe ejecutar una operación completa y devolver un resultado consistente.	AnalysisService.analizar_apk() analiza y arma la respuesta completa
Manejo de Errores	Se usan excepciones tipadas y HTTPException con código y detalle en vez de TRY...CATCH SQL.	ExternalServiceError, HTTPException(status_code=502, detail=...)

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

3.3 Declaración de Funciones Definidas por el Usuario

Las funciones puras (métodos privados con guion bajo inicial) se utilizan para encapsular cálculos o lógica reutilizable que no modifica datos externos.

Regla	Descripción	Ejemplo (AnzenCore)
Prefijo	Ninguno obligatorio; nombres descriptivos en snake_case. Los métodos privados llevan guion bajo inicial.	_hash_password, _normalize_level
Propósito	Se usan para cálculo o validación que no modifica datos ni estado.	_mask_secret(value), _max_severity(findings)
Validación	Se usan para verificar condiciones antes de una operación.	ReportService._normalize_level() valida 'nivel' antes de guardar

Funciones de Tabla

Regla	Descripción	Ejemplo (AnzenCore)
Prefijo	Ninguno obligatorio; se nombran según los datos que procesan/listan.	_detect_insecure_http, _build_structure_artifacts
Propósito	Analizan una colección de datos y devuelven una lista de resultados.	ApkAnalyzer._detect_possible_secrets() retorna list[ApkFinding]

3.4 Declaración de Triggers

AnzenCore no usa Triggers: la integridad de datos se refuerza con restricciones CHECK a nivel de columna (status, severity, severity_max, export_format, component_type) y la auditoría se aplica a nivel de servicio Python (validaciones antes de insertar y timestamps automáticos con default now()).

Regla	Descripción	Ejemplo (AnzenCore)
Restricciones (CHECK)	Reemplaza la integridad que un Trigger validaría en tiempo de ejecución, aplicada directamente por	CHECK en apk_scans.status, apk_findings.severity,

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

	Postgres.	report_exports.export_format, apk_components.component_type
Validación previa	Equivalente a BEFORE INSERT: se valida antes de insertar.	ReportService.create_report() valida que exista el user_id antes de insertar
Post-procesamiento	Equivalente a AFTER INSERT/UPDATE: pasos adicionales tras una operación exitosa.	SupabaseModel._upgrade_password() re-hashea la contraseña tras un login exitoso con hash legacy
Auditoría	Se registran timestamps (default now()) en cada inserción relevante.	created_at, started_at, finished_at en apk_scans

3.5 Control de versiones de código fuente

Cada modificación relevante se registra mediante un commit de Git siguiendo esta convención:

Título	Descripción
Formato	Control de versiones con Git. Commits descriptivos y versionado semántico (MAJOR.MINOR.PATCH) para artefactos publicables (ej. extensión VS Code).
Descripción	El repositorio se aloja en GitHub, con GitHub Actions ejecutando la pipeline de DevSecOps (tests, SonarQube, Snyk) en cada push/PR.

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

4. Procedimientos y Funciones definidos por el Usuario.

El estándar dicta que el flujo de trabajo completo de AnzenCore debe estar encapsulado en clases de servicio Python para garantizar la integridad, el rendimiento y la reutilización del código, ya que no existen procedimientos ni funciones a nivel de base de datos.

4.1 Procedimientos Almacenados (SPs) - Flujo Transaccional

Los métodos de servicio manejan las operaciones de análisis y registro (crear, consultar, actualizar). Al no existir transacciones SQL explícitas, cada método debe validar antes de llamar a Supabase y propagar excepciones tipadas si algo falla.

Módulo/Acción	Método (equivalente a SP)	Parámetros de Entrada Clave	Propósito Principal
Análisis APK	AnalysisService.analizar_apk	file_bytes, file_name	Analiza un APK y arma la respuesta con hallazgos y resumen.
Análisis código fuente	AnalysisService.analizar_codigo_fuente	file_bytes, file_name	Analiza código fuente filtrando solo los hallazgos aplicables.
Análisis URL	AnalysisService.analizar_url	url	Escanea una URL en busca de vulnerabilidades remotas.
Análisis GitHub	AnalysisService.analizar_repo_github	repo_url	Delega el análisis de calidad de un repositorio a un servicio externo.
Autenticación	SupabaseModel.authenticate	username, password	Verifica credenciales (PBKDF2, con fallback a SHA-256 legacy).
Reportes	ReportService.create_	user_id, dispositivo,	Valida y registra un hallazgo en la

	report	vulnerabilidad, nivel, descripcion	tabla vulnerabilidades.
Escaneo de APK (dashboard)	ApkScanService.analyze	file_bytes	Ejecuta el análisis local y arma el payload para apk_scans.

4.2 Funciones Definidas por el Usuario (UDFs) - Lógica y Consultas

Las funciones puras se utilizan para cálculos y validaciones (severidad máxima, enmascarado de secretos, normalización de nivel) que no implican modificación de datos.

4.2.1 Funciones Escalares

Módulo/Acción	Función Requerida	Parámetros de Entrada Clave	Valor de Retorno	Propósito Principal
Seguridad	SupabaseModel._hash_password	password: str	str (hash hex)	Aplica PBKDF2-HMAC-SHA256 con 260 000 iteraciones.
Seguridad (legacy)	SupabaseModel._hash_password_legacy	password: str	str (hash hex)	Hash SHA-256 heredado, usado solo como fallback de login.
Validación	ReportService._normalize_level	level: str	str	Normaliza y valida el nivel de severidad de un reporte.
Enmascarado	ApkAnalyzer._mask_secret	value: str	str	Enmascara un secreto detectado antes de mostrarlo como evidencia.
Severidad	ApkAnalyzer._max_severity	findings: list	str	Devuelve la severidad máxima de una lista de hallazgos.

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

4.2.2 Funciones de Tabla

Módulo/Acción	Función Requerida	Parámetros de Entrada Clave	Propósito Principal
Deduplicación	ApkAnalyzer._deduplicate_findings	findings: list	Elimina hallazgos duplicados por tipo/título/archivo/evidencia.
Listados	FolderAnalyzer.analyze_folder	project_name, file_data	Devuelve métricas y code smells por archivo de una carpeta.
Reportes	SupabaseModel.get_apk_scans	user_id	Devuelve el historial de escaneos APK de un usuario.
Reportes (global)	SupabaseModel.get_all_apk_scans	(ninguno)	Devuelve métricas globales de todos los escaneos, para el dashboard.

5. Beneficios

- La aplicación de validaciones Pydantic y chequeos explícitos en la capa de servicio para procesos críticos, como el análisis de un APK, asegura que las reglas de negocio (tipos de análisis válidos, niveles de severidad) se cumplan antes de persistir en Supabase. Esto es la máxima garantía contra datos inconsistentes.
- El uso de funciones puras y clases de servicio para encapsular la lógica evita que los desarrolladores repitan código entre la API (FastAPI) y el dashboard (Streamlit). Esto reduce la probabilidad de errores lógicos o inconsistencias entre ambos frontends.
- La modularidad de las clases de servicio y la nomenclatura estándar (snake_case, sufijo Service) hacen que cualquier desarrollador pueda entender, modificar o añadir nuevos tipos de análisis con un impacto mínimo en el código existente.

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

- Gracias a los timestamps de auditoría (created_at, started_at, finished_at) registrados en cada escaneo, se puede identificar rápidamente cuándo ocurrió un análisis y su duración, acelerando la depuración.
- El uso de consultas Supabase optimizadas (filtros eq/gt, orden por fecha, límites de tamaño de archivo) para reportes críticos asegura que el sistema cumpla con los requerimientos no funcionales de rendimiento al analizar proyectos grandes.
- El registro de hallazgos en la tabla vulnerabilidades y de escaneos en apk_scans proporciona evidencia auditable de qué se analizó, cuándo y con qué resultado, cumpliendo con los requisitos de seguridad y control interno.
- La definición de consultas dedicadas para el dashboard (get_all_apk_scans, get_apk_findings, get_apk_artifacts) permite extraer información analítica de manera eficiente para la toma de decisiones.

6. Conclusiones

- La arquitectura de AnzenCore establece una estructura clara y coherente que facilita la segregación de responsabilidades entre rutas (routes), servicios (services), modelos (models) y esquemas (schemas).
- La aplicación de esquemas Pydantic y validaciones explícitas garantiza un mecanismo robusto de integridad de datos, asegurando el cumplimiento de reglas críticas de negocio (niveles de severidad válidos, tipos de análisis permitidos) y la

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

trazabilidad de los escaneos y hallazgos registrados en Supabase.

- Se ha programado con éxito el flujo de trabajo completo de análisis de vulnerabilidades (APK, código fuente, URL y repositorios GitHub) a través de clases de servicio dedicadas, lo que automatiza el proceso y reduce errores humanos.
- El uso de servicios y consultas Supabase optimizadas asegura que el sistema pueda manejar de manera eficiente el volumen de escaneos y hallazgos, cumpliendo con los requerimientos no funcionales de rendimiento.
- El uso de nombres descriptivos y reglas sistemáticas de nomenclatura (PEP 8) ha contribuido a una mejor identificación de variables y componentes, lo que reduce la posibilidad de errores y acelera los tiempos de mantenimiento.
- La definición de consultas dedicadas para reportes y métricas permite una extracción de datos eficiente, facilitando el análisis estratégico de los hallazgos de seguridad y el cumplimiento de los objetivos analíticos del proyecto.